Software Requirements Specification (SRS)

BLOCKCHAIN VOTING SYSTEM

CRYPTOGRAPHIC E-VOTING & DECENTRALIZED LEDGER

Document Version: 1.0

Prepared By: Aditya Yadav

Institution: TRCAC (Thakur Ramnarayan College of Arts & Commerce)

Department: B.Sc. Computer Science

Academic Year: 2026–2027

Date: August 2026

Status: Final Draft

Table of Contents

Sr. No.	Main Section	Description
1	**Introduction**	Introduces the project, its purpose, scope, audience, and references.
2	**Overall Description**	Provides an overview of the system, modules, workflow, and operating environment.
3	**User Roles & Characteristics**	Describes different user roles and their responsibilities within the system.
4	**Functional Requirements**	Defines all core features and functions that the system must perform.
5	**Non-Functional Requirements**	Specifies performance, security, reliability, usability, scalability, and maintainability requirements.
6	**System Constraints**	Lists technical limitations, assumptions, and development constraints.
7	**External Interface Requirements**	Describes the user interface, page structure, and interactions with external components.
8	**Data Requirements**	Defines database entities, data structures, relationships, and storage requirements.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the Blockchain Voting System (BVS). The system is designed to digitise, secure, and streamline the complete election cycle — encompassing voter registration, candidate nomination, 1-click encrypted voting, Proof-of-Work block mining, and verifiable cryptographic receipts.

This document serves as the primary reference for developers, project managers, QA engineers, and institutional administration throughout the system development lifecycle.

1.2 Project Scope

The Blockchain Voting System is a web-based multi-role application that centralises the end-to-end election operations. The system replaces manual, paper-based, and centralized database processes with a unified, role-driven platform that ensures data integrity, operational visibility, and tamper-evident vote recording.

The system covers the following operational domains:

- User profile and voter eligibility management
- Election scheduling and lifecycle state management
- Candidate registration and manifesto management
- 1-Click encrypted ballot casting with AES-256 GCM encryption
- Asymmetric digital signature signing via ECDSA SECP256R1
- Proof-of-Work block mining and binary Merkle Tree computation
- Downloadable QR code receipts and hash verification desk
- AI Fraud Radar tracking vote velocity bursts and duplicate IP clusters
- Real-time election standings and audit trail logging

1.3 Intended Audience

Audience	Usage
Software Development Team	Design, develop, and integrate system modules per specifications
Project Manager	Track deliverables, milestones, and scope boundaries
QA / Testing Team	Derive test cases and acceptance criteria
Institutional Administration	Validate that requirements reflect institutional election needs
Election Officer / Auditor	Primary end-user; manages day-to-day elections and reviews audit logs
System Administrator	Understand roles, access levels, and security policies

1.4 Definitions and Abbreviations

Term / Acronym	Definition
BVS	Blockchain Voting System
PoW	Proof-of-Work — consensus algorithm for computational block mining
ECDSA	Elliptic Curve Digital Signature Algorithm (SECP256R1 curve)
AES-256 GCM	Advanced Encryption Standard with Galois/Counter Mode authenticated encryption
SHA-256	Secure Hash Algorithm 256-bit cryptographic digest function
Merkle Tree	Cryptographic binary hash tree aggregating all transactions within a block
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
JWT	JSON Web Token — stateless cryptographic bearer token for authentication
FK / PK	Foreign Key / Primary Key — database relational identifiers
TRCAC	Thakur Ramnarayan College of Arts & Commerce

1.5 References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications
- ISO/IEC 25010:2011: Systems and Software Quality Requirements and Evaluation
- OWASP Security Guidelines for Web Applications
- NIST Special Publication 800-38D (GCM Block Cipher Mode)
- SEC 2: Recommended Elliptic Curve Domain Parameters (SECP256R1)
- Tailwind CSS and React 19 Component Library Documentation
- FastAPI Asynchronous Web Framework Documentation

2. Overall Description

2.1 System Context

The BVS is a centralised, browser-based platform accessible via standard browsers on desktop, tablet, and mobile devices. The system integrates multiple operational modules under a single authentication layer, with all relational application data persisted in a database and vote transactions secured in an immutable custom Python Blockchain ledger.

2.2 System Modules

#	Module	Responsibility
1	User Management	Handles user registration, authentication, and role assignment
2	Election Management	Manages election creation, scheduling, and status lifecycle
3	Candidate Management	Registers candidates, stores manifestos, party, and avatars
4	Voting & Encryption	Voter casts AES-256 encrypted ballots with ECDSA signatures
5	Blockchain Engine	Packages transactions, mines PoW blocks, and generates Merkle roots
6	Receipt & Verification	Generates QR receipts and verifies receipt hashes on the ledger
7	AI Fraud Radar	Detects vote velocity bursts (<2s), IP clusters, and risk score
8	Observer & Analytics	Real-time vote counts, turnout percentages, and Recharts charts
9	Reports & Audit	Immutable audit logs, system metrics, and export to CSV

2.3 System Workflow

The following sequence describes the standard end-to-end operational flow within the Blockchain Voting System:

1. Administrator logs in and creates user accounts, assigning appropriate roles (Admin, Voter, Observer).
2. Voters and Observers log in using their credentials and receive a stateless 24-hour JWT token.
3. Administrator creates new elections, sets start/end dates, and adds candidate profiles with manifestos.
4. Administrator uploads voter CSV files to bulk-enroll eligible voters.
5. Voters browse active elections, review candidate manifestos, and select their preferred candidate.
6. The system encrypts the vote payload using AES-256 GCM and signs the transaction with single-use ECDSA keypair.
7. Blockchain Engine validates single-vote rule, queues transaction, and triggers Proof-of-Work block mining.
8. Block is mined with SHA-256 difficulty target, computes binary Merkle root, and persists to ledger.
9. Voter receives a downloadable QR receipt containing receipt hash, block index, and transaction hash.
10. AI Fraud Radar continuously analyzes submission timestamps and flags anomaly risk scores (0-100%).
11. Observers and Administrator review live Recharts standings and export official audit reports.

3. User Roles & Characteristics

3.1 User Roles Summary

Role	System Responsibilities
Administrator	Full system access; manages users, roles, elections, candidates, bulk CSV voter enrollment, and platform configuration
Voter	Views active elections, reviews candidates, casts encrypted votes, downloads QR receipts, and verifies hashes
Observer / Auditor	Monitors live standings, inspects block explorer, audits blockchain integrity, and views AI fraud alerts

4. Functional Requirements

4.1 User Management

1. The system shall provide a secure login mechanism using email/username and password credentials with encrypted password storage (bcrypt).
2. The system shall allow the Administrator to create, view, update, and deactivate user accounts.
3. Each user shall be assigned exactly one role from the predefined role set (Admin, Voter, Observer).
4. The system shall restrict access to modules and pages based on the authenticated user's assigned role.
5. The system shall maintain a security log recording every user login and logout event with timestamp and IP address.

4.2 Election Management

1. The system shall allow Administrator to create elections with title, description, start time, and end time.
2. The system shall allow Administrator to transition election status between Draft, Active, and Closed.
3. The system shall automatically filter and display active elections to registered voters.
4. The system shall prevent voting on elections that are in Draft or Closed status.

4.3 Candidate Management

1. The system shall allow Administrator to register candidates with name, party/affiliation, manifesto, and avatar URL.

2. The system shall store candidate-wise profile details and manifestos.
3. The system shall display candidate profiles on the voter ballot with manifesto preview popups.

4.4 Job Posting & Voting Application (Voting Module)

1. The system shall allow eligible voters to select a candidate and cast an encrypted vote.
2. The system shall encrypt the vote payload using AES-256 GCM authenticated symmetric encryption before storage.
3. The system shall digitally sign each vote transaction using a single-use ECDSA SECP256R1 keypair.
4. The system shall strictly enforce a single vote per registered voter per election, blocking duplicate attempts.

4.5 Blockchain Engine & Block Mining

1. The system shall package queued vote transactions into blocks meeting SHA-256 Proof-of-Work difficulty targets.
2. The system shall compute a cryptographic binary Merkle Root from all transaction hashes in each block.
3. The system shall maintain an immutable block chain linked by cryptographic SHA-256 previous block hashes.
4. The system shall provide a 1-click ledger audit function that verifies chain integrity and detects tampering.

4.6 Receipt Generation & Verification

1. The system shall generate a downloadable QR code receipt containing receipt hash, block index, and transaction hash.
2. The system shall allow voters and auditors to verify any receipt hash against the public blockchain ledger.
3. The system shall confirm whether a verified receipt hash is recorded in a valid mined block.

4.7 AI Fraud Radar & Anomaly Detection

1. The system shall track vote submission timestamps and client IP addresses in real time.
2. The system shall detect velocity bursts (<2 seconds between votes) and flag automated bot submissions.
3. The system shall detect duplicate IP clusters exceeding configurable threshold limits.
4. The system shall calculate a composite Fraud Risk Index (0-100%) and assign a threat level (Low, Medium, High, Critical).

4.8 Reports & Analytics

1. The system shall display real-time live standings, vote counts, leading candidates, and turnout percentages.
2. The system shall render interactive bar and pie charts using Recharts for visual analytics.
3. The system shall allow export of election results and audit logs to CSV format.

5. Non-Functional Requirements

5.1 Security

- All user passwords shall be stored using a strong cryptographic hashing algorithm (bcrypt with salt rounds = 10). Plain-text storage is strictly prohibited.
- All HTTP communications shall be enforced over HTTPS (TLS 1.2 or higher).
- Vote payloads shall be encrypted with AES-256 GCM authenticated encryption.
- Transaction authenticity shall be enforced via ECDSA SECP256R1 asymmetric key pair signatures.
- Session tokens shall be stateless JWT Bearer tokens expiring after 24 hours.
- All page endpoints shall enforce server-side role-based access control. Client-side restrictions alone are not sufficient.
- The system shall log all login attempts (successful and failed) to the Security Log.

5.2 Performance

- Standard page loads and API responses shall complete within 500 milliseconds under normal network conditions.
- Proof-of-Work block mining shall complete within 2 seconds under difficulty target = 2.
- The system shall support at least 500 concurrent active users without performance degradation.
- Real-time WebSocket vote broadcasts shall be pushed to client dashboards within 1 second.

5.3 Usability

- The interface shall require no specialist technical training. A new user shall be able to complete core tasks within their role after a 2-minute orientation.
- All forms shall display inline validation feedback before or immediately upon submission.
- All data tables shall support sorting, column filtering, and pagination with a configurable page size.
- The application shall be fully functional on modern browsers — Chrome, Firefox, Edge, Safari.

5.4 Reliability and Availability

- The system shall target 99.9% uptime during active election windows.
- Immutable blockchain ledger structure guarantees zero vote record deletion or unauthorized alteration.
- The system shall implement graceful error handling — unexpected errors shall display a user-friendly message rather than a raw stack trace.

5.5 Scalability

- The application architecture shall support horizontal scaling to accommodate increased load during peak voting hours.
- The database schema shall be extensible to support future modules without breaking existing functionality.

5.6 Maintainability

- All source code shall follow a consistent coding standard and include inline documentation for complex logic.
- The application shall be structured using a modular MVC architectural pattern to facilitate independent module updates.

6. System Constraints

6.1 Constraints

- The system shall be built using React 19 and Tailwind CSS as the primary frontend UI framework.
- The system shall use Python 3.12 with FastAPI as the backend REST API framework.
- The system shall use a relational database management system (SQLite / PostgreSQL or equivalent).
- The custom blockchain Proof-of-Work difficulty target shall be configured to 2 leading zeros for lightweight execution.
- Voter CSV enrollment uploads shall be limited to CSV format with a maximum file size of 10 MB.
- Vote attempts are strictly limited to one vote per registered voter per election, enforced by unique compound database constraints.

6.2 Assumptions

- Each user is assigned a single role (Admin, Voter, or Observer). Multi-role access per user account is not required in version 1.0.
- Network connectivity is stable within the operating infrastructure.
- User devices (computers and tablets) will have modern evergreen browsers installed.
- The operating environment will provide secure server infrastructure with protected environment variables (SECRET_KEY, VOTE_ENCRYPTION_KEY).
- Registered voters possess unique email addresses and valid login credentials.

7. External Interface Requirements

7.1 UI Design Philosophy

The system interface shall be designed to professional enterprise standards, prioritising clarity, navigational consistency, and operational efficiency.

- Clean, structured layout with a fixed top navigation bar and a collapsible left sidebar for module navigation.
- Dashboard-first approach — the default landing view provides an at-a-glance operational summary for the logged-in role.
- Data-heavy pages shall use well-styled responsive tables with column sorting, pagination, and search/filter capability.
- All forms shall use inline validation feedback to minimise input errors.
- Status indicators shall use coloured badge combinations so information is never conveyed by colour alone.
- The overall aesthetic should be modern and professional — clean whites, dark modes, navy accents, and teal highlights.

7.2 Page Inventory and Role Mapping

#	Page Name	Accessible By	Primary Purpose
1	Login / Register	All (unauthenticated)	Authenticate users and route to dashboard
2	Dashboard	All Roles (post-login)	Centralised operational overview tailored per role
3	User Management	Admin only	Full control over user accounts and role assignments
4	Election Management	Admin only	Create, schedule, activate, and close elections
5	Candidate Studio	Admin only	Register and manage candidate profiles and manifestos
6	Bulk Voter Import	Admin only	Upload CSV files to bulk-enroll voters
7	Voter Dashboard	Voter only	Active elections list, ballot casting, and voting history
8	QR Vote Receipt	Voter only	Download and print cryptographic QR vote receipt
9	Receipt Verification	All Roles	Public verification desk to check receipt hash against ledger
10	Blockchain Explorer	All Roles	Inspect mined blocks, transactions, and trigger chain audit
11	Observer Dashboard	Observer, Admin	Live Recharts standings, vote percentages, and turnout
12	AI Fraud Radar	Admin, Observer	Real-time velocity monitoring and anomaly risk score

7.3 Page-by-Page Design Specification

Page 1 — Login / Register

The Login page is the entry point of the application. It shall be a centred, modern card layout. The card shall contain the application logo and name at the top, followed by Email and Password fields, quick demo role login buttons, and a Login button.

Accessible By: All users (unauthenticated)

Interactions: Form submission, role selection, field validation, redirect on success

Page 2 — Dashboard

The Dashboard is the primary landing page for all authenticated roles. The layout consists of a fixed top navbar, a collapsible left sidebar, and a main content area dynamically tailored to the logged-in role. Admin sees summary stat cards, a blockchain mining health widget, and quick-action buttons; Voters see Active Ballots; Observers see live election standings.

Accessible By: All Roles

Interactions: Navigation, stat card drill-downs, quick actions

Page 3 — User Management (Admin Only)

This page is restricted to the Administrator. It presents a full-page data table of all registered users with columns for name, email, role, and account status. A prominent Add User button opens a modal with the new user form (Name, Email, Password, Role selector). Row-level action buttons allow editing user details, reassigning roles, and deactivating accounts.

Accessible By: Admin

Interactions: Add user modal, edit user, deactivate with confirmation

Page 4 — Election Management

A searchable, filterable table of all elections with columns for title, start date, end date, candidate count, and status badges. The Admin can create elections, activate voting, and close elections. Each row provides an Edit and View Candidates option.

Accessible By: Admin

Interactions: Create election modal, status toggle, filter by status

Page 5 — Candidate Studio

A list of all registered candidates with their party affiliation, manifesto, and avatar image. A prominent Add Candidate button opens a form. Individual candidate rows provide an Edit option and ballot preview.

Accessible By: Admin

Interactions: Add candidate, edit candidate, delete candidate, view ballot preview

Page 6 — Bulk Voter Import

The Bulk Voter Import page allows administrators to upload a CSV file containing voter records (email, username, password). The system validates the CSV in real-time, displays a preview, and imports accounts in a single batch with progress tracking.

Accessible By: Admin

Interactions: Upload CSV file, review preview, confirm batch import

Page 7 — Voter Dashboard

Shows active elections available to the voter. Displays candidate cards with photos, party emblems, and a View Manifesto button. The voter can select a candidate and cast a 1-click encrypted ballot with confirmation dialog.

Accessible By: Voter

Interactions: View candidates, read manifesto, cast encrypted vote

Page 8 — QR Vote Receipt

A cryptographic receipt page rendered immediately after voting. Displays the receipt hash, block index, timestamp, and a downloadable high-resolution QR code receipt for voter verification.

Accessible By: Voter

Interactions: Download receipt, print receipt, copy receipt hash

Page 9 — Receipt Verification Desk

A public verification desk allowing voters and auditors to input any receipt hash. The system queries the blockchain ledger and confirms whether the vote was mined into a valid block without exposing voter identity.

Accessible By: All Roles

Interactions: Input receipt hash, query blockchain, view verification status

Page 10 — Blockchain Explorer

A full-page ledger browser displaying all mined blocks with block index, timestamp, previous hash, current hash, nonce, Merkle root, and transaction count. Includes a 1-click ledger audit button to detect chain tampering.

Accessible By: All Roles

Interactions: Inspect block details, search transactions, trigger chain audit

Page 11 — Observer Dashboard

An analytics and transparency page for Observers and Admin. Displays summary cards — Total Registered Voters, Total Votes Cast, Voter Turnout Percentage, Leading Candidates. Interactive Recharts render real-time vote distribution.

Accessible By: Observer, Admin

Interactions: View live stats, toggle chart visualizers, export CSV report

Page 12 — AI Fraud Radar

A security monitoring dashboard tracking vote velocity bursts (<2s), duplicate IP clusters, and double-voting attempts. Computes a composite Threat Index (0-100%) and categorizes election health.

Accessible By: Admin, Observer

Interactions: View velocity graph, inspect flagged IP clusters, review threat score

8. Data Requirements

All persistent data is stored in a relational database. Tables are normalised to Third Normal Form (3NF) to minimise redundancy. Foreign key constraints enforce referential integrity across all relationships.

8.1 ROLE

Field Name	Data Type	Key	Description
role_id	INT	PK	Auto-increment primary key
role_name	VARCHAR(50)	—	Admin, Voter, Observer

8.2 USER

Field Name	Data Type	Key	Description
user_id	INT	PK	Auto-increment primary key
name	VARCHAR(100)	—	Full name of the user
email	VARCHAR(100)	UNIQUE	Login credential and unique identifier
password	VARCHAR(255)	—	Hashed password (bcrypt)
role_id	INT	FK → ROLE	Defines the access role assigned to this user
is_active	BOOLEAN	—	Account status — active or deactivated

8.3 ELECTION

Field Name	Data Type	Key	Description
election_id	INT	PK	Auto-increment primary key
title	VARCHAR(200)	—	Title of the election
description	TEXT	—	Full election description and rules
status	VARCHAR(20)	—	Draft / Active / Closed
start_time	DATETIME	—	Scheduled election opening timestamp
end_time	DATETIME	—	Scheduled election closing timestamp
created_by	INT	FK → USER	Admin user who created the election

8.4 CANDIDATE

Field Name	Data Type	Key	Description
candidate_id	INT	PK	Auto-increment primary key
election_id	INT	FK → ELECTION	Election this candidate is running in
name	VARCHAR(100)	—	Full name of the candidate
party	VARCHAR(100)	—	Party name or organizational affiliation
manifesto	TEXT	—	Candidate manifesto and platform promises
avatar_url	VARCHAR(255)	—	URL to candidate portrait image
vote_count	INT	—	Aggregated tally of confirmed votes

8.5 VOTE

Field Name	Data Type	Key	Description
vote_id	INT	PK	Auto-increment primary key
user_id	INT	FK → USER	Voter account who cast the ballot
election_id	INT	FK → ELECTION	Election being voted on
candidate_id	INT	FK → CANDIDATE	Selected candidate
voter_hash	VARCHAR(64)	—	Anonymous SHA-256 voter fingerprint
encrypted_vote	TEXT	—	AES-256 GCM encrypted ballot payload
tx_hash	VARCHAR(64)	UNIQUE	Unique cryptographic transaction hash
block_index	INT	—	Index of the mined block containing the vote
receipt_hash	VARCHAR(64)	UNIQUE	Verifiable receipt lookup hash

8.6 BLOCK

Field Name	Data Type	Key	Description
block_id	INT	PK	Auto-increment primary key
block_index	INT	UNIQUE	Sequential block index in chain (0 = Genesis)
timestamp	FLOAT	—	Unix timestamp when block was mined
previous_hash	VARCHAR(64)	—	SHA-256 hash of the preceding block
hash	VARCHAR(64)	UNIQUE	SHA-256 hash of current block meeting PoW target
nonce	INT	—	Proof-of-Work counter satisfying difficulty
merkle_root	VARCHAR(64)	—	Binary Merkle root aggregating block transactions
signature	TEXT	—	System ECDSA digital signature

8.7 TRANSACTION

Field Name	Data Type	Key	Description
tx_id	INT	PK	Auto-increment primary key
tx_hash	VARCHAR(64)	UNIQUE	SHA-256 transaction digest
block_index	INT	FK → BLOCK	Block that confirmed this transaction
election_id	INT	FK → ELECTION	Associated election ID
voter_hash	VARCHAR(64)	—	Anonymous voter hash
encrypted_vote	TEXT	—	AES-256 encrypted payload
signature	TEXT	—	Voter ECDSA digital signature

8.8 AI_ANOMALY

Field Name	Data Type	Key	Description
anomaly_id	INT	PK	Auto-increment primary key
election_id	INT	FK → ELECTION	Associated election ID
anomaly_type	VARCHAR(50)	—	Velocity Spike / IP Cluster / Double Vote
ip_address	VARCHAR(45)	—	Client IP address flagged
risk_score	FLOAT	—	Computed threat score (0.0 to 100.0)
timestamp	DATETIME	—	Timestamp when anomaly was flagged

8.9 AUDIT_LOG

Field Name	Data Type	Key	Description
log_id	INT	PK	Auto-increment primary key
user_id	INT	FK → USER	User session being recorded
user_email	VARCHAR(100)	—	User email recorded at event time
action	VARCHAR(100)	—	Action performed (LOGIN, CAST_VOTE, CREATE_ELECTION)
details	TEXT	—	Contextual details and metadata
ip_address	VARCHAR(45)	—	Client IP address
timestamp	DATETIME	—	Event timestamp

8.10 Entity Relationship Summary

The relationships between core database entities are as follows:

- **ROLE (1) → USER (many):** Each user is assigned one role; a role may be assigned to many users.
- **USER (1) → VOTE (many):** Each voter can cast votes across multiple distinct elections (enforcing a unique compound constraint on user_id, election_id).
- **ELECTION (1) → CANDIDATE (many):** An election contains multiple running candidates.
- **ELECTION (1) → VOTE (many):** An election accumulates confirmed votes from registered voters.
- **ELECTION (1) → AI_ANOMALY (many):** An election generates real-time AI anomaly detection logs.
- **BLOCK (1) → TRANSACTION (many):** A mined block contains multiple confirmed vote transactions.
- **USER (1) → AUDIT_LOG (many):** Each user session generates immutable security audit log entries.